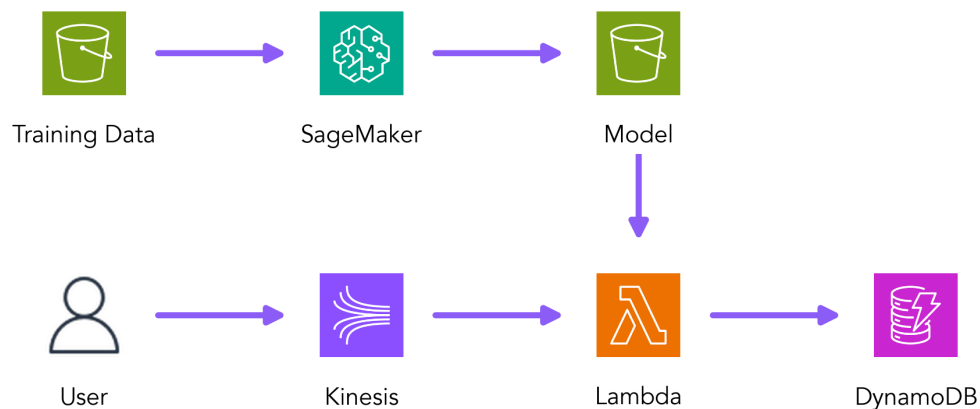




Credit Risk Modeling on AWS

Real-Time Application Credit Scoring

Credit Scoring Architecture



What This Guide Covers:

- Real-time credit scoring on AWS
- WOE Logistic Regression scorecard
- SageMaker Local Mode for model training
- LocalStack for local AWS development and testing
- CloudFormation for credit scorecard deployment



Overview

This chapter demonstrates a cloud-native credit scoring system using AWS services, implemented locally with LocalStack for development and testing. The architecture processes loan applications in real-time, applies a scorecard model, and stores approved applications for auditability.

Key Learning Objectives

By the end of this guide chapter, you will understand:

- How native AWS services can help in building a credit scoring system
- Weight of Evidence (WOE) transformation for credit risk modeling
- Selecting optimal cutoff for credit decisioning
- Local development workflow using LocalStack and SageMaker Local Mode
- Infrastructure as Code (IaC) with CloudFormation templates for deployment

Architecture Components

The credit scoring system consists of two main flows: **Training** and **Inference**.

Training Flow

Model Training Pipeline

The training pipeline implements a credit scorecard using Weight of Evidence (WOE) transformation combined with logistic regression:

1. **Data Preparation:** Historical credit data stored in S3 (training data bucket)
2. **Model Training:** SageMaker trains a WOE + Logistic Regression pipeline
3. **Scorecard Conversion:** Logistic coefficients are converted to a points-based scorecard using the standard credit scoring convention
4. **Model Artifacts:** Trained model and metadata are saved to S3 (model artifacts store)

Scorecard Formula:

$$\text{Score} = \text{offset} + \text{factor} \times (\beta_0 + \beta_1 \cdot \text{WOE}_1 + \beta_2 \cdot \text{WOE}_2 + \dots)$$

Where:

- $\text{factor} = \frac{\text{pts_double_odds}}{\ln(2)} = \frac{20}{\ln(2)} \approx 28.85$
- $\text{offset} = \text{target_score} - \text{factor} \times \ln(\text{target_odds}) = 600 - 28.85 \times \ln(30)$
- Target score: 600, Target odds: 30, Points to double odds: 20

Inference Flow

Real-Time Scoring Pipeline

The inference pipeline processes loan applications in real-time:

1. **User:** Submits loan application (amount, income, credit history, etc.)
2. **Kinesis Data Stream:** Ingests application data as streaming events
3. **Lambda Function:**
 - Triggered by Kinesis events
 - Loads trained scorecard model from S3
 - Applies WOE transformation and calculates credit score
 - Compares score against cutoff (502 points)
4. **DynamoDB:** Stores approved applications (score ≥ 502) for audit and analysis

Decision Logic:

$$\text{Decision} = \begin{cases} \text{APPROVED} & \text{if Score} \geq 502 \\ \text{DECLINED} & \text{if Score} < 502 \end{cases}$$

Credit Scorecard Methodology

Weight of Evidence (WOE) Transformation

WOE is a powerful technique for transforming categorical and continuous variables in credit scoring:

Definition:

$$\text{WOE}_i = \ln \left(\frac{\% \text{ of Bads}_i}{\% \text{ of Goods}_i} \right)$$

Alternatively, we can calculate it as $\text{WOE}_i = \ln \left(\frac{\% \text{ of Goods}_i}{\% \text{ of Bads}_i} \right)$ thus reversing the sign of the WOE.

Why use WOE in application credit scoring?

- **Economic relationships:** Establishes monotonic relationships with log-odds of default
- **Handles missing and special values:** Can assign WOE to missing and special categories
- **Interpretability:** Positive WOE indicates higher risk of default, negative WOE indicates lower risk of default

Pipeline:

Raw Features $\xrightarrow{\text{WOE Transform}}$ WOE Values $\xrightarrow{\text{Logistic Regression}}$ Log-Odds $\xrightarrow{\text{Scorecard}}$ Credit Score

Optimal Cutoff Determination

The cutoff is determined by maximizing the separation between goods and bads using the Kolmogorov-Smirnov Statistic (also Youden Index):

Strategy: Find threshold where difference between % bads captured and % goods captured is maximum:

$$\text{Cutoff}^* = \arg \max_{\text{threshold}} (\text{Share}_{\text{bads}}(\text{threshold}) - \text{Share}_{\text{goods}}(\text{threshold}))$$

For our trained model:

- **Optimal cutoff:** 502 points
- **Approval rate:** ~60%
- **Bad rate in approved:** ~15%
- **Gini coefficient:** 0.5847 (strong discriminatory power)

Local Development with LocalStack

Why LocalStack?

LocalStack enables local AWS development without cloud costs or internet dependency:

- **Cost-effective:** No AWS charges during development
- **Fast iteration:** Instant deployment and testing
- **Reproducible:** Consistent environment across team members
- **Offline development:** Works without internet connection
- **CI/CD friendly:** Easy integration with automated testing

Services Emulated:

- S3 (training data and model storage)
- Kinesis Data Streams (real-time ingestion)
- Lambda (scoring logic)
- DynamoDB (approved applications storage)
- CloudFormation (infrastructure deployment)

LocalStack includes a number of services that are not available in the AWS Free Tier, such as Kinesis Data Streams and DynamoDB. For SageMaker, we are using the Local Mode which is a Docker container that simulates the AWS SageMaker environment without cloud costs.

Code Examples

Training with SageMaker Python SDK

This example shows how to use SageMaker's Local Mode to train a model in a Docker container on your local machine, simulating the AWS SageMaker environment without cloud costs.

```

from sagemaker.local import LocalSession
from sagemaker.sklearn.estimator import SKLearn

# Use LocalSession for local training
sagemaker_session = LocalSession()
sagemaker_session.config = {'local': {'local_code': True}}

# Define the estimator
sklearn_estimator = SKLearn(
    entry_point='train.py',
    source_dir='training',
    role='arn:aws:iam::123456789012:role/SageMakerRole',
    instance_type='local',
    framework_version='1.2-1',
    py_version='py3',
    sagemaker_session=sagemaker_session,
    hyperparameters={
        'target_score': 600,
        'target_odds': 30,
        'pts_double_odds': 20
    }
)

# Train the model locally
sklearn_estimator.fit({'training': 's3://bucket/data'})

```

Credit Scorecard Training Script

This example shows how to train a credit scorecard model using FastWoe for Weight of Evidence transformation and scikit-learn's Logistic Regression. The pipeline is then converted to a points-based scorecard using the standard credit scoring formula.

```

import numpy as np
import pandas as pd
from fastwoe import FastWoe
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

# Load credit data
df = pd.read_csv('data/credit_example.csv')
X = df.drop('Good_Bad', axis=1)
y = df['Good_Bad']

# Create WOE + Logistic Regression pipeline
pipeline = Pipeline([
    ('woe', FastWoe()),
    ('logistic', LogisticRegression(penalty=None, solver='lbfgs'))
])

# Train the model
pipeline.fit(X, y)

# Convert to scorecard
target_score = 600
target_odds = 30
pts_double_odds = 20

```

```

factor = pts_double_odds / np.log(2)
offset = target_score - factor * np.log(target_odds)

# Calculate scores
scores = offset - factor * pipeline.decision_function(X)

```

Real-Time Scoring with Lambda

This Lambda function is triggered by Kinesis events (a service for real-time streaming data), loads the trained scorecard pipeline, calculates credit scores for incoming applications, and stores approved applications (score $\geq$ cutoff) in DynamoDB, a key-value store for storing approved applications.

```

import json

import boto3
import joblib

# Load trained model
pipeline = joblib.load('model/scorecard_pipeline.joblib')
metadata = json.load(open('model/scorecard_metadata.json'))

def lambda_handler(event, context):
    # Parse Kinesis record
    for record in event['Records']:
        payload = json.loads(base64.b64decode(record['kinesis']['data']))

        # Extract features
        features = pd.DataFrame([
            'loan_amount': payload['amount'],
            'income': payload['income'],
            'debt_to_income': payload['dti'],
            # ... other features
        ])

        # Calculate credit score
        score = calculate_score(features, pipeline, metadata)

        # Decision logic
        if score >= metadata['cutoff']:
            store_approved_application(payload, score)
            return {'decision': 'APPROVED', 'score': score}
        else:
            return {'decision': 'DECLINED', 'score': score}

```

Sending Applications to Kinesis

Next, we pass loan applications to the Kinesis stream for real-time processing. Each application includes loan amount, income, debt-to-income ratio, and other application features.

```
import json
from datetime import datetime

import boto3

kinesis = boto3.client('kinesis', endpoint_url='http://localhost:4566')

# Generate loan application
application = {
    'application_id': f'app-{datetime.now().timestamp()}',
    'amount': 25000,
    'income': 65000,
    'debt_to_income': 0.35,
    'credit_history_length': 5,
    'num_credit_lines': 3,
    'timestamp': datetime.now().isoformat()
}

# Send to Kinesis
response = kinesis.put_record(
    StreamName='credit-scoring-stream',
    Data=json.dumps(application),
    PartitionKey=application['application_id']
)

print(f"Sent application {application['application_id']}")
```

Querying Approved Applications

In this step, we query the DynamoDB table for approved applications.

```
import boto3

dynamodb = boto3.resource('dynamodb', endpoint_url='http://localhost:4566')
table = dynamodb.Table('ApprovedApplications')

# Scan approved applications
response = table.scan()

for item in response['Items']:
    print(f"ID: {item['application_id']}")
    print(f"Score: {item['credit_score']}")
    print(f"Amount: ${item['loan_amount']:,.0f}")
    print(f"Income: ${item['income']:,.0f}")
    print(f"Timestamp: {item['timestamp']}")
```

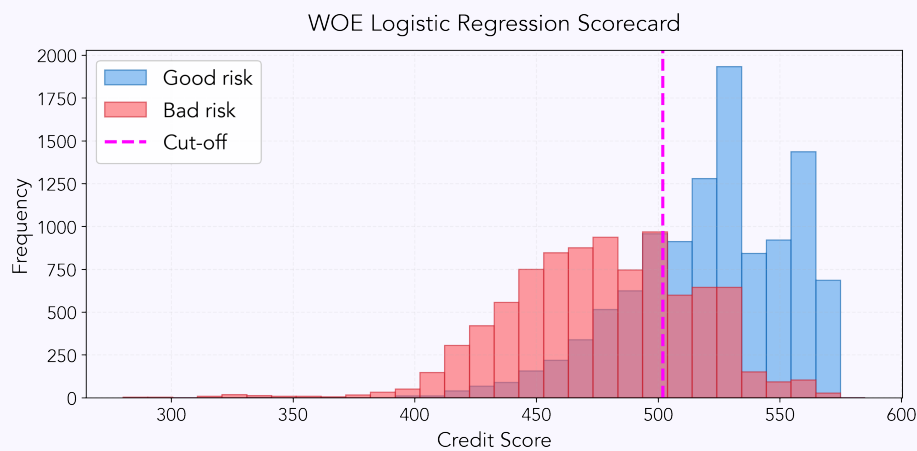
Model Performance

Scorecard Performance Metrics

The trained scorecard demonstrates strong predictive power:

Metric	Value
Gini Score	0.5847
Optimal Cutoff	502 points
Approval Rate	60.2%
Bad Rate (Approved)	14.8%

Score Distribution:



The histogram shows the separation between good (blue) and bad (red) credit risks. The fuchsia dashed line indicates the optimal cutoff at 502 points, which maximizes the difference between the percentage of bads captured and goods captured.

Key Takeaways

What You've Learned

- **Credit Scorecard:** WOE transformation + Logistic Regression provides interpretable, industry-standard credit scoring
- **Real-Time Architecture:** Kinesis + Lambda enables low-latency credit decisioning at scale
- **Local Development:** LocalStack provides a complete AWS emulation for rapid iteration
- **Infrastructure as Code:** CloudFormation templates ensure reproducible deployments

Other Use Cases in this Series

- Credit Card Limit Management Orchestration
- Credit Fraud Detection with PD Model Components
- Customer Chat Sentiment Classification